

Louis Dionne (ldionne.2@gmail.com)
Richard Smith (richard@metafoo.co.uk)
Nina Ranns (dinka.ranns@gmail.com)
Daveed Vandevoorde (daveed@edg.com)

More constexpr containers

R0: Original proposal, presented in Albuquerque 2017.

EWG approved the direction: SF: 11 | F: 12 | N: 2 | A: 0 | SA: 0

R1: Provided initial wording.

EWG approved the proposal as presented: SF: 23 | F: 11 | N: 0 | A: 0 | SA: 0

R2: Implemented core review comments.

R3: Extends proposal for non-transient allocations.

R4 (this revision): Tighten wording for "construct" and "destruct"

Introduction and motivation

Variable size container types, like `std::vector` or `std::unordered_map`, are generally useful for runtime programming, and therefore also potentially useful in constexpr computations. This has been made clear by some recent experiments such as the [Constexpr ALL the things!](#) presentation (and its companion paper [P0810R0](#) published in the pre-Albuquerque mailing) by Ben Deane and Jason Turner, in which they build a compile-time JSON parser and JSON value representation using constexpr. Amongst other things, the lack of variable size containers forces them to use primitive fixed-size data structures in the implementation, and to parse the input JSON string twice; once to determine the size of the data structures, and once to parse the JSON into those structures.

We also expect variable size containers to be a necessity in the reflection and metaprogramming APIs that will emerge from the work in SG-7, which decided that the preferred direction for a standard solution would involve constexpr-like computation. For example, querying the template arguments of a class type might look something like:

```
std::vector<std::metainfo> args = std::meta::get_template_args(reflexpr(T));
```

Non-transient allocation

Earlier versions of this paper ([P0784r1](#) and [P0784r2](#)) proposed the changes need for constexpr destructors and for so-called "transient constexpr allocations". Transient constexpr allocations are dynamic memory allocations occurring during a constexpr evaluation that are deallocated before that evaluation completes.

What about dynamically allocated constexpr storage that hasn't been deallocated by the time evaluation completes? We could that, but there are really compelling use cases where this might be desirable. E.g., this could be the basis for a more flexible kind of "string literal" class. We therefore propose that a non-transient constexpr allocation be a valid result for a constexpr variable initializer if:

- the result of evaluating the initializer is an object with a nontrivial constexpr destructor, and
- evaluating that destructor would be a valid core constant expression and would deallocate all the *non-transient* allocations produced by the evaluation of *expr*.

Furthermore, we specify that an attempt to deallocate a non-transiently allocated object by any other means results in undefined behavior. (Note that this is unlikely because the object pointing to the allocated storage is immutable.)

For example:

```
#include <memory>
#include <new>
using namespace std;
template<typename T> struct S: allocator<T> {
    T *ps;
    int sz;
    template<int N> constexpr S(T (&p)[N])
        : sz{N}
        , ps{this->allocate(N)} {
        for (int k = 0; k<N; ++k) {
            new(this->ps+k) T{p[k]};
        }
    }
    constexpr ~S() {
        for (int k = 0; k<this->sz; ++k) {
            (this->ps+k)->T::~~T();
        }
        this->deallocate(this->ps, this->sz);
    }
};

constexpr S<char> str("Hello!");
// str ends up pointing to a static array
// containing the string "Hello!".
```

The constructor constexpr evaluation in this example is successful, producing an S object that points to a non-transient constexpr allocation. The constexpr evaluation of the destructor would also be successful and would deallocate the non-transient allocation. The non-transient allocation is therefore promoted to static storage.

Implementation experience

So far, this has not been implemented. However, based on preliminary discussion with implementers working on Clang, MSVC and EDG, no blockers that would make this feature unimplementable or prohibitively expensive to implement have been identified at the moment.

Wording changes

Note 1: These are cumulative changes: They include the changes EWG approved for P0784r1, and build on top of that.

Note 2: The following changes enable "constexpr destructors". See further down for allocation-related changes.

Change in [basic.types] paragraph 10.5.1:

— it has a ~~trivial~~constexpr destructor,

Change in [expr.const] paragraph 2:

An expression *e* is a *core constant expression* unless the evaluation of *e*, following the rules of the abstract machine (4.6), would evaluate one of the following expressions:

— this (8.1.2), except in a constexpr function ~~or~~, a constexpr constructor, or a constexpr destructor that is being evaluated as part of e;

— an invocation of a function other than a constexpr constructor or destructor for a literal class, or a constexpr function, ~~or an implicit invocation of a trivial destructor (15.4)~~ [Note: Overload resolution (16.3) is applied as usual — end note];

— an invocation of an undefined constexpr function ~~or~~, an undefined constexpr constructor, or an undefined constexpr destructor;

— an invocation of an instantiated constexpr function ~~or~~, a constexpr constructor, or a constexpr destructor that fails to satisfy the requirements for a constexpr function ~~or~~, a constexpr constructor, or a constexpr destructor (10.1.5);

— an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either

— it is initialized with a constant expression, ~~or~~

— its lifetime began within the evaluation of e, or;

— for an *id-expression* that refers to a data member of a constexpr variable x with static storage duration, the evaluation occurs during the invocation of a constexpr destructor for x.

— modification of an object (8.18, 8.2.6, 8.3.2) unless it is applied to a non-volatile lvalue of literal type that refers to ~~non-volatile object whose lifetime began within the evaluation of e~~

— a non-volatile object whose lifetime began within the evaluation of e, or

— a non-volatile subobject of a constexpr variable x with static storage duration and the modification occurs in an invocation of a constexpr destructor for x

Add new paragraph after [expr.const] paragraph 2:

An object is said to have *constant destruction* if:

— it is of a non-class type, or

— the type of that object has a constexpr destructor and for a hypothetical expression e whose only effect is to destroy that object, e is a core constant expression.

Change in [dcl.constexpr] paragraph 2:

A constexpr specifier used in the declaration of a function that is not a constructor or a destructor declares that function to be a *constexpr function*. Similarly, a constexpr specifier used in a constructor declaration declares that constructor to be a *constexpr constructor*. A constexpr specifier used in a destructor declaration declares that destructor to be a *constexpr destructor*.

Insert new paragraph after [dcl.constexpr] paragraph 4:

The definition of a constexpr destructor shall satisfy the following requirements:

— it shall not be virtual

— the class shall not have any virtual base classes;

— its *function-body* shall satisfy the requirements for a *function-body* of a `constexpr` function;

Change in [dcl.constexpr] paragraph 6:

If the instantiated template specialization of a `constexpr` function template or member function of a class template would fail to satisfy the requirements for a `constexpr` function ~~or, a `constexpr` constructor, or a `constexpr` destructor~~, that specialization is still a `constexpr` function ~~or, a `constexpr` constructor, or a `constexpr` destructor~~, even though a call to such a function cannot appear in a constant expression. If no specialization of the template would satisfy the requirements for a `constexpr` function ~~or, a `constexpr` constructor, or a `constexpr` destructor~~ when considered as a non-template function ~~or, a constructor, or a destructor~~, the template is ill-formed, no diagnostic required.

Change in [dcl.constexpr] paragraph 8:

The `constexpr` specifier has no effect on the type of a `constexpr` function ~~or, a `constexpr` constructor, or a `constexpr` destructor~~.

Change in [dcl.constexpr] paragraph 9:

In any `constexpr` variable declaration, the full-expression of the initialization shall be a constant expression (8.20). **A `constexpr` variable shall have constant destruction.**

Change in [class.dtor] paragraph 1:

Each decl-specifier of the decl-specifier-seq of a destructor declaration (if any) shall be `friend`, `inline`, ~~or~~ `virtual`, **or `constexpr`**.

Add after [class.dtor] paragraph 9:

The defaulted destructor is a `constexpr` destructor if

— it is a trivial destructor, or

— it is not virtual and all the destructors it invokes are `constexpr` destructors

Note 3: The following changes enable some "constexpr new-expressions".

Modify [expr.new] paragraph 10

An implementation is allowed to omit a call to a replaceable global allocation function (21.6.2.1, 21.6.2.2). When it does so, the storage is instead provided by the implementation or provided by extending the allocation of another *new-expression*.

~~The implementation may extend the allocation of a new-expression *e1* to provide storage for a new-expression *e2* if the following would be true were the allocation not extended:~~

~~— the evaluation of *e1* is sequenced before the evaluation of *e2*, and~~

~~— *e2* is evaluated whenever *e1* obtains storage, and~~

~~— both *e1* and *e2* invoke the same replaceable global allocation function, and~~

~~— if the allocation function invoked by *e1* and *e2* is throwing, any exceptions thrown in the evaluation of either *e1* or *e2* would be first caught in the same handler, and~~

~~— the pointer values produced by *e1* and *e2* are operands to evaluated delete-expressions, and~~

— the evaluation of *e2* is sequenced before the evaluation of the *delete-expression* whose operand is the pointer value produced by *e1*.

[*Example:*

...

— *end example*]

Add new paragraph after [expr.new] paragraph 10

During an evaluation of a constant expression, a call to an allocation function is always omitted. [*Note:* Only *new-expressions* that would otherwise result in a call to a replaceable global allocation function can be evaluated in constant expressions (see [expr.const]). — *end note*]

Add new paragraph after [expr.new] paragraph 10

The implementation may extend the allocation of a *new-expression* *e1* to provide storage for a *new-expression*. *e2* if the following would be true were the allocation not extended:

— the evaluation of *e1* is sequenced before the evaluation of *e2*, and

— *e2* is evaluated whenever *e1* obtains storage, and

— both *e1* and *e2* invoke the same replaceable global allocation function, and

— if the allocation function invoked by *e1* and *e2* is throwing, any exceptions thrown in the evaluation of either *e1* or *e2* would be first caught in the same handler, and

— the pointer values produced by *e1* and *e2* are operands to evaluated *delete-expressions*, and

— the evaluation of *e2* is sequenced before the evaluation of the *delete-expression* whose operand is the pointer value produced by *e1*.

[*Example:*

...

— *end example*]

Change in [expr.const] paragraph 2:

— a *new-expression* (8.3.4);

— a *new-expression* (8.3.4), unless the selected allocation function is a replaceable global allocation function (21.6.2.1, 21.6.2.2);

— a *delete-expression* (8.3.5);

Note 4: The following changes enable the use of the default allocator in constant expressions.

Add a new paragraph after [expr.const] paragraph 2:

For the purposes of determining whether an expression is a core constant expression, the evaluation of a call to a member function of `std::allocator<T>` as defined in `_allocator.members_`, where `T` is a literal type, is permitted even if the actual evaluation of such a call would otherwise fail the requirements for a core constant expression. Similarly, the evaluation of a call to a member function

of `std::allocator_traits<std::allocator<T>>` as defined in `_allocator.traits,members_`, is a valid core constant expression unless

- for a call to the member `construct`, the second argument, of type `T*`, does not point to storage allocated with `std::allocator<T>` or the evaluation of the underlying constructor call is not a core constant expression, or
- for a call to the member `destroy`, the second argument, of type `T*`, does not point to storage allocated with `std::allocator<T>` or the evaluation of the underlying destructor call is not a core constant expression.

Note 5: The following changes enable non-transient allocations.

Change bullet (6.2) of [expr.const] paragraph 6 from:

- if the value is of pointer type, it contains the address of an object with static storage duration, the address past the end of such an object (8.5.6), the address of a function, or a null pointer value, and

to:

- if the value is of pointer type, it contains
 - the address of an object with static storage duration or the address past the end such an object (8.5.6), or
 - the address of a function, or
 - a null pointer value, or,
 - if the prvalue under consideration initializes a `constexpr` variable, the address of a non-transient `constexpr` allocation (as defined below) or the address past the end (`_expr.add_`) of such an allocation, and

Add to end of [expr.const] paragraph 6:

A non-transient `constexpr` allocation is an object of dynamic storage duration allocated and not deallocated during the initialization of a `constexpr` variable of class type or array of class type, such that evaluation of that variable's destructor(s) immediately after that initialization would deallocate the object. Deallocating a non-transient `constexpr` allocation before the (implicit) invocation of the associated `constexpr` variable's destructor results in undefined behavior.